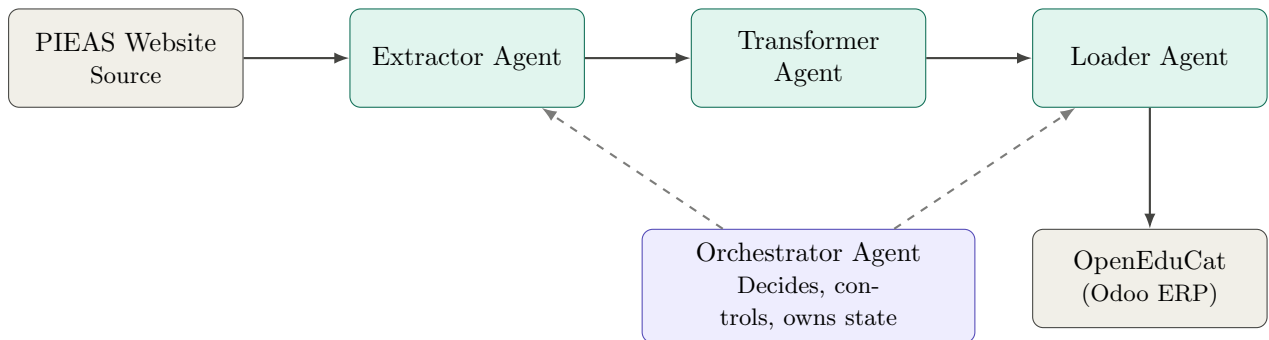


Agentic Multi-Agent Data Synchronization System

PIEAS Website → OpenEduCat ERP

A Multi-Agent Approach to One-Time Migration and
Continuous Data Synchronization



Project Report

Course Assignment — Multi-Agent Systems / AI Agents

July 28, 2026 (Revision 2 — simplified 4-agent design)

Contents

1	Introduction	2
1.1	Background: Odoo	2
1.2	Background: OpenEduCat	2
1.3	Why move from a standalone LMS to OpenEduCat	2
2	Problem Statement	4
2.1	Aim	4
3	Proposed Solution: Multi-Agent Architecture	5
3.1	Design principle: one agent, one clear job	5
3.2	Architecture diagram	5
3.3	Agent responsibilities	5
3.4	Optional extension: Validation Agent	6
4	Phases: Bulk Migration vs. Incremental Sync	7
4.1	Two cycles within incremental sync	7
5	Change and Deletion Detection	8
5.1	Change detection: Method 1 (considered, not used) — Boolean dirty flag	8
5.2	Change detection: Method 2 (recommended and adopted) — Timestamp water- mark	8
5.3	Deletion detection	9
5.4	What to do with a detected deletion	9
6	SQL Queries Used in the System	10
6.1	State / mapping table (owned exclusively by the Orchestrator)	10
6.2	Lookup before writing (the upsert decision)	10
6.3	Recording a new mapping after a successful create	10
6.4	Advancing the watermark after a successful change-cycle run	10
6.5	Fetching the full ID list for the deletion cycle	10
6.6	Finding missing records (deletion detection)	11
6.7	Archiving a record removed from PIEAS	11
7	Loading Data into OpenEduCat: Methods Considered	12
8	Design Questions Raised and Their Resolutions	13
9	Conclusion	15

1 Introduction

1.1 Background: Odoo

Odoo is an open-source, modular Enterprise Resource Planning (ERP) platform, originally released in 2005 (as TinyERP, later OpenERP). It is built on a Python backend with a PostgreSQL database, and exposes an internal Object-Relational Mapping (ORM) layer that all business logic is written against. Rather than being one monolithic application, Odoo is a collection of independent *modules* (“apps”) — CRM, Accounting, Inventory, HR, and many more — that share a single underlying database and can be installed incrementally as an organization’s needs grow.

Two properties of Odoo matter directly for this project:

- **The ORM enforces correctness.** Every write to the database that goes through Odoo’s ORM (`create`, `write`, `search`) also triggers computed fields, validation constraints, and related-record bookkeeping. Writing directly to the PostgreSQL tables bypasses all of this and risks producing inconsistent, half-formed records — silently, with no error raised.
- **External access is first-class.** Odoo exposes an XML-RPC API (and, in Odoo 19, a newer JSON-2 API) that lets any external program authenticate and call the same ORM methods remotely. This is the mechanism this project’s Loader Agent uses to write into OpenEduCat.

1.2 Background: OpenEduCat

OpenEduCat is an open-source Student Information System / education-management suite built as a set of modules *on top of* Odoo. It reuses Odoo’s ORM, database engine, and API mechanisms, and adds education-specific models such as `op.student`, `op.faculty`, `op.course`, and `op.batch`. Because it inherits Odoo’s architecture, everything true of Odoo above — ORM-enforced correctness, XML-RPC/REST access — is also true of OpenEduCat.

OpenEduCat’s Community edition is free, open-source (LGPL-3), and fully self-hostable, covering the core student information system (admissions, attendance, exams, fees, library) with no per-user or per-student fee. A paid Enterprise tier exists, but only adds optional extras (AI suite, full LMS, proctored quizzes, hostel/transport management, vendor support, managed hosting) on top of the same free core — it is not required to run OpenEduCat, and is not required for this project.

1.3 Why move from a standalone LMS to OpenEduCat

Institutions typically outgrow a self-built, standalone LMS/portal for a consistent set of reasons, which motivate this project:

- **Maintenance burden.** A custom-built system is usually understood by only a small number of people, with thin documentation, and every new requirement means new bespoke code.
- **Data silos.** A standalone LMS typically only knows about courses and grades. An ERP-based system such as OpenEduCat can unify student records, faculty data, and (if desired) finance and HR under a single database, rather than several disconnected systems.
- **Ecosystem and longevity.** OpenEduCat is maintained externally and benefits from the wider Odoo module ecosystem, active releases, and community support, rather than depending on one institution’s in-house development capacity.

- **Standardized integration.** OpenEduCat's ORM/API access means future integrations (attendance devices, payment gateways, other campus systems) can be built against a documented interface instead of custom code specific to the legacy system.

2 Problem Statement

PIEAS currently maintains its own website / system holding student, faculty, and course-related data. The institution wants to adopt OpenEduCat as its Student Information System going forward. This creates two distinct problems that must both be solved:

1. **One-time migration.** All existing data on the PIEAS website (students, faculty, courses, and related records) must be transferred into OpenEduCat before it can be used as the system of record.
2. **Ongoing synchronization.** After the migration, PIEAS continues to be used and its data continues to change (e.g., a quiz mark is updated, a new student is admitted, a student record is removed). OpenEduCat must reflect these changes automatically, without manual re-export, and without requiring PIEAS to be decommissioned on day one.

A further constraint shapes the whole design: **PIEAS exposes no API and no webhooks.** It cannot notify anything when its data changes. Any detection of change, and any detection of deletion, has to happen by actively checking PIEAS, not by PIEAS pushing updates out.

2.1 Aim

To design and build an **agentic, multi-agent system** that:

- performs a one-time bulk migration of all PIEAS data into OpenEduCat,
- subsequently runs continuously, detecting and propagating only the records that changed on PIEAS, and
- separately detects records that were *removed* from PIEAS, so OpenEduCat does not keep stale records forever —

using a small set of cooperating agents with clearly separated, non-overlapping responsibilities, rather than a single monolithic script or agents whose jobs quietly duplicate one another.

3 Proposed Solution: Multi-Agent Architecture

The system is split into four agents. Earlier design iterations of this project considered a fifth “Monitor” agent dedicated to change detection, but this was deliberately removed: its job (deciding when to check PIEAS, and comparing results against known state) overlapped entirely with the Orchestrator’s job, and it would have required either duplicating PIEAS access outside the Extractor or holding a second copy of synced data — both undesirable. The Orchestrator absorbs that responsibility instead, since it already owns the state needed to do the comparison.

3.1 Design principle: one agent, one clear job

- **Only the Extractor ever talks to PIEAS.** It is a pure fetcher: given an instruction, it goes and gets exactly that data, and nothing more. It holds no memory between calls and makes no decisions about what the data means.
- **Only the Orchestrator makes decisions.** It decides what needs fetching, when, and what to do with the result. It is the only agent that reads or writes the local state store.
- **Transformer and Loader are stateless workers.** They do exactly one job each on whatever they are handed, regardless of which cycle triggered them.

3.2 Architecture diagram

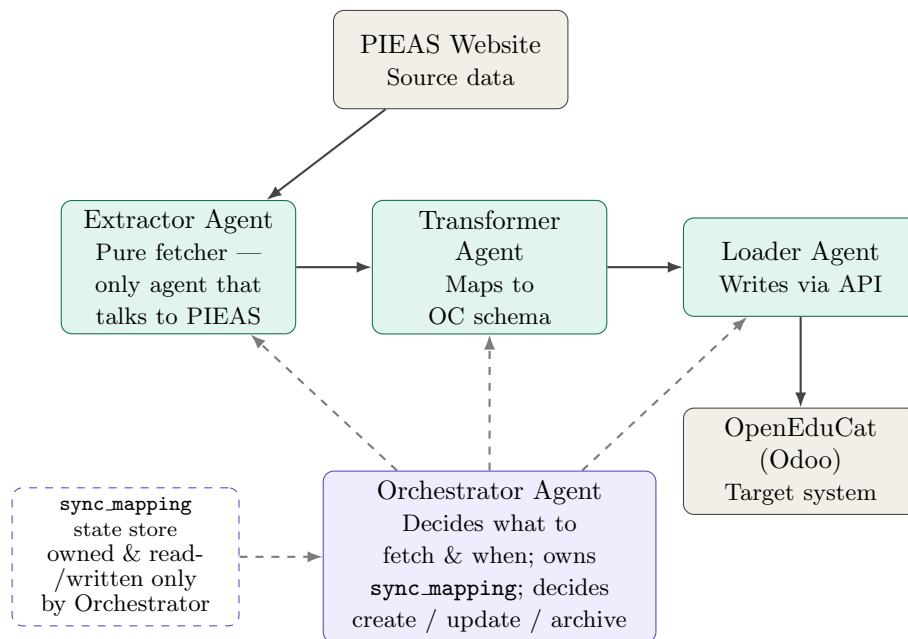


Figure 1: Simplified four-agent architecture. Solid arrows show the data pipeline (PIEAS → OpenEduCat). Dashed arrows show instructions and state ownership — all controlled by the Orchestrator, which is the only agent with a memory of prior syncs.

3.3 Agent responsibilities

Agent	Responsibility
Extractor Agent	Pure fetcher; the <i>only</i> agent that ever contacts PIEAS. Takes one of two instructions from the Orchestrator: (1) fetch rows where <code>last_updated</code> is newer than the watermark, or (2) fetch the full current list of PIEAS record IDs (no filtering, IDs only, not full records). Makes no comparisons and holds no state between calls.
Transformer Agent	Maps PIEAS's raw schema onto OpenEduCat's data model (e.g. <code>op.student</code> , <code>op.faculty</code>). Implemented as a pure function (record in, OpenEduCat-shaped record out); identical logic regardless of which cycle triggered it or how many records it is given.
Loader Agent	Writes transformed records into OpenEduCat via XML-RPC (or REST, see Section 7). Executes exactly the action the Orchestrator decided: <code>create</code> (new record), <code>write</code> (update an existing one), or <code>write</code> with <code>active=False</code> (archive a record no longer on PIEAS).
Orchestrator Agent	The only decision-making agent. Owns and is the sole reader/writer of the <code>sync_mapping</code> state store. On every scheduled run, decides which fetch to request from the Extractor, compares the result against <code>sync_mapping</code> , classifies each record as new / changed / unchanged / missing, and dispatches the corresponding action to Transformer and Loader. Also owns retry and error-logging logic, and the switch from BULK mode (state store empty) to SYNC mode (state store populated).

3.4 Optional extension: Validation Agent

A fifth, strictly optional agent can be added once the core four-agent pipeline is working: after a Loader write, it re-reads the affected record back from OpenEduCat and confirms it matches what was sent (e.g. no orphaned foreign keys, field values match). This is a quality-assurance addition, not part of the core decision loop, and does not change any of the responsibilities above — it observes after the fact rather than participating in the extract/decide/write flow.

4 Phases: Bulk Migration vs. Incremental Sync

The system operates in two phases that share the same pipeline but differ in scope, triggering, and the Loader’s write behaviour.

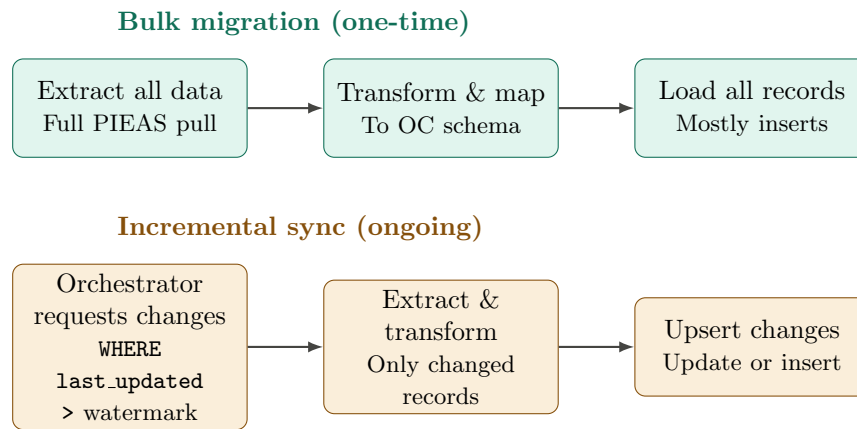


Figure 2: Bulk migration runs once and processes every record; incremental sync runs on a schedule and processes only what changed. Both phases use the same Extractor, Transformer, and Loader — only the Orchestrator’s instruction to the Extractor differs.

	Bulk migration	Incremental sync
Runs	Once, at project start	Repeatedly, on a schedule (e.g. every 15–60 min), forever after
Scope	Every PIEAS record	Only records changed since the last run
Trigger	Orchestrator sees an empty state store	Timestamp watermark comparison, owned by Orchestrator
Extractor instruction	“Fetch everything, paginate until exhausted”	“Fetch rows where <code>last_updated > watermark</code> ”
Loader operation	Mostly <code>create</code>	Upsert: <code>write</code> if mapped, else <code>create</code>
State store role	Write-only — built from scratch	Read-heavy — every record’s fate depends on a lookup by the Orchestrator

4.1 Two cycles within incremental sync

Incremental sync is itself not one cycle but two, both driven by the Orchestrator, both using the same Extractor, but asking for different things and running on different schedules. The first catches *changes*; the second catches *deletions* — a case the first cycle structurally cannot detect, since a deleted row is absent rather than changed, and a query filtered on `last_updated` never sees a row that no longer exists.

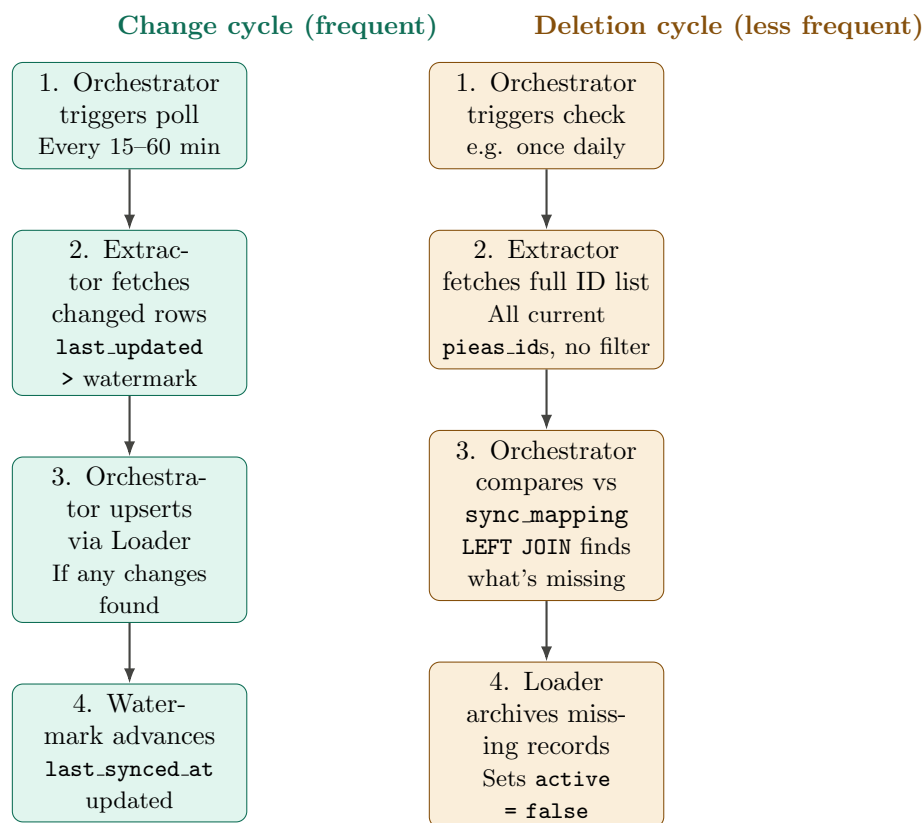


Figure 3: Both cycles are entirely owned and sequenced by the Orchestrator. The Extractor is called by both, with a different instruction each time; it does not decide anything or hold any state between calls.

5 Change and Deletion Detection

PIEAS exposes no webhooks and no change notifications, so both change detection and deletion detection must be performed actively, by the Orchestrator, using data the Extractor fetches on request.

5.1 Change detection: Method 1 (considered, not used) — Boolean dirty flag

Add a `sync_pending` column to PIEAS’s tables; a trigger sets it to 1 whenever a row is inserted or updated; the Extractor queries only rows where the flag is set, then resets it after a successful sync.

Problem: if a row changes again *while* it is being synced, blindly resetting the flag to 0 afterward can silently drop that second change (a race condition). It also assumes only one process ever resets the flag.

5.2 Change detection: Method 2 (recommended and adopted) — Timestamp watermark

Instead of a boolean that gets mutated, keep a `last_updated` timestamp on each PIEAS row (most databases maintain this automatically, or it is trivial to add via a trigger) and a single `last_synced_at` watermark that the Orchestrator owns. A change is anything newer than the watermark:

```
1 SELECT *
```

```

2 FROM students
3 WHERE last_updated > :last_synced_at
4 ORDER BY last_updated ASC;

```

Listing 1: Change-cycle query, run by the Extractor on the Orchestrator’s instruction

This avoids the race condition entirely – there is no flag to reset, only a watermark that advances forward after a run completes – and it supports more than one consumer reading the same change set safely. **This is the recommended method**, adopted for the system.

5.3 Deletion detection

The change-cycle query above can never detect a deletion: it only returns rows that still exist, so a row that has vanished is simply never seen. Detecting a deletion therefore requires a structurally different check, run less often (e.g. daily rather than every poll), in two steps:

Step 1 — Extractor fetches a full, unfiltered ID list (only IDs, not full records, to keep this cheap):

```

1 SELECT pieas_id
2 FROM students;

```

Listing 2: Deletion-cycle fetch, run by the Extractor on the Orchestrator’s instruction

Step 2 — Orchestrator compares that list against its own sync_mapping table (this comparison happens locally; PIEAS is not queried again):

```

1 SELECT sm.pieas_id, sm.openeducat_id
2 FROM sync_mapping sm
3 LEFT JOIN students s ON s.pieas_id = sm.pieas_id
4 WHERE sm.entity_type = 'student'
5 AND s.pieas_id IS NULL;

```

Listing 3: Orchestrator’s local comparison to find missing records

A `LEFT JOIN` is used specifically because it keeps every row from `sync_mapping` even when there is no matching row on the PIEAS side — that absence of a match (`IS NULL`) is exactly the deletion signal being looked for.

5.4 What to do with a detected deletion

Three options were considered:

1. **Archive (recommended and adopted).** Set Odoo’s `active` field to `False`. The record is hidden from normal views but historical data (grades, past enrollment) is preserved.
2. **Hard delete.** Actually remove the record. Rejected as the default: irreversible, and risky if a PIEAS “deletion” was actually a rename or a temporary status change.
3. **Flag for manual review.** The Orchestrator logs the missing record instead of acting automatically; a human confirms before anything is written. A safer alternative worth mentioning even though archiving was adopted as the default.

6 SQL Queries Used in the System

6.1 State / mapping table (owned exclusively by the Orchestrator)

```

1 CREATE TABLE sync_mapping (
2     id SERIAL PRIMARY KEY,
3     pieas_id VARCHAR(50) NOT NULL,
4     openeducat_id INTEGER NOT NULL,
5     entity_type VARCHAR(50) NOT NULL,      -- 'student', '
        faculty', 'course', ...
6     content_hash VARCHAR(64),
7     last_synced_at TIMESTAMP DEFAULT NOW(),
8     UNIQUE (pieas_id, entity_type)
9 );

```

Listing 4: Sync state / mapping table schema

6.2 Lookup before writing (the upsert decision)

```

1 SELECT openeducat_id
2 FROM sync_mapping
3 WHERE pieas_id = %s
4 AND entity_type = 'student';

```

Listing 5: Orchestrator: check whether a PIEAS record has already been synced, before instructing the Loader

If a row is returned, the Orchestrator instructs the Loader to **write** (update) that `openeducat_id`; if no row is returned, it instructs a **create**, followed by a new mapping row.

6.3 Recording a new mapping after a successful create

```

1 INSERT INTO sync_mapping (pieas_id, openeducat_id, entity_type,
2     last_synced_at)
3 VALUES (%s, %s, 'student', NOW())
4 ON CONFLICT (pieas_id, entity_type)
5 DO UPDATE SET openeducat_id = EXCLUDED.openeducat_id,
6     last_synced_at = NOW();

```

Listing 6: Recording the PIEAS to OpenEduCat mapping after an insert

6.4 Advancing the watermark after a successful change-cycle run

```

1 UPDATE sync_state
2 SET last_synced_at = NOW()
3 WHERE table_name = 'students';

```

Listing 7: Updating the global sync watermark once a change-cycle run completes

6.5 Fetching the full ID list for the deletion cycle

```

1 SELECT pieas_id
2 FROM students;

```

Listing 8: Extractor's deletion-cycle fetch instruction

6.6 Finding missing records (deletion detection)

```
1 SELECT sm.pieas_id, sm.openeducat_id
2 FROM sync_mapping sm
3 LEFT JOIN students s ON s.pieas_id = sm.pieas_id
4 WHERE sm.entity_type = 'student'
5 AND s.pieas_id IS NULL;
```

Listing 9: Orchestrator’s local comparison against sync_mapping

6.7 Archiving a record removed from PIEAS

Executed by the Loader, via the OpenEduCat ORM (XML-RPC `write`), not raw SQL — shown here in SQL form only to illustrate the effect:

```
1 UPDATE op_student
2 SET active = false
3 WHERE id = %s;    -- openeducat_id, from sync_mapping
```

Listing 10: Conceptual effect of the Loader’s archive write on the OpenEduCat side

7 Loading Data into OpenEduCat: Methods Considered

Three ways of writing into OpenEduCat were discussed. All three ultimately reach the same PostgreSQL database, but only two of them go through Odoo’s ORM, which is what enforces computed fields, validation, and related-record bookkeeping.

Method	How it works	Verdict
XML-RPC (Odoo’s built-in ORM access)	Authenticate once via the common endpoint to obtain a <code>uid</code> , then call ORM methods (<code>create</code> , <code>write</code> , <code>search</code>) remotely through the <code>object</code> endpoint. Built into every self-hosted Odoo/OpenEduCat install with nothing to enable.	Recommended. Guaranteed to exist on any self-hosted instance; goes through the ORM so validation and computed fields behave correctly; well documented across the whole Odoo ecosystem.
OpenEduCat REST API	Plain HTTP requests (POST/PUT) against documented endpoints (e.g. <code>/api/v1/students</code>), authenticated with an API key or OAuth2. Also goes through the same underlying ORM.	Viable and arguably cleaner, <i>but</i> availability may depend on the specific OpenEduCat edition/hosting setup and should be confirmed on the actual instance before relying on it. Treated as a fallback/alternative transport for the Loader Agent.
Direct PostgreSQL writes	Bypass Odoo entirely; connect straight to the database and run <code>INSERT/UPDATE</code> against the underlying tables.	Rejected. Skips computed fields, business-rule validation, and related-table bookkeeping (e.g. linked <code>res.partner</code> contact records); risks half-formed, inconsistent records that fail <i>silently</i> — no error is raised, the UI simply shows stale or missing data later; schema is a moving target across versions. Acceptable only for read-only reporting queries, never for the sync pipeline’s writes.

On self-hosting: XML-RPC is not technically restricted to self-hosted Odoo — the restriction is a licensing decision Odoo imposes on its own hosted *Odoo Online* product, where external API access is gated to the Custom pricing plan. A self-hosted instance has no such gatekeeper. Self-hosting OpenEduCat does not mean writing OpenEduCat from scratch: it is an existing, free, open-source Odoo module (`openeducat_erp`) that is installed into a self-hosted Odoo Community instance, the same way any Odoo app is installed — no OpenEduCat code needs to be written for this project.

Decision: the Loader Agent is built against **XML-RPC** as the primary transport, since it is guaranteed to be present, safest with respect to data correctness, and best documented for a project on a fixed timeline.

8 Design Questions Raised and Their Resolutions

The table below summarizes the key design questions worked through while designing this system, and the conclusion reached for each – intended as a decision log for anyone extending this project.

Question	Resolution
How should PIEAS data be matched to existing OpenEduCat records rather than duplicated?	Match on a locally-held <code>pieas_id</code> → <code>openeducat_id</code> mapping table, looked up before every write, rather than querying OpenEduCat live on each check.
A student’s data (e.g. quiz marks) can change at any time – won’t a simple unique-key match miss that?	Yes – matching alone only tells you a record <i>exists</i> , not that it <i>changed</i> . Change detection is a separate step, resolved via timestamp watermarking.
Should change detection use a dirty-flag column, as in some CDC systems?	Considered, but rejected in favour of a timestamp watermark due to the reset race condition (Section 5).
Can we load data by writing directly to OpenEduCat’s PostgreSQL tables instead of using the ORM/API?	Technically possible but rejected – bypasses computed fields, validation, and related-record creation, and fails silently (Section 7).
Self-hosted XML-RPC vs. OpenEduCat’s own REST service – which should the Loader use?	XML-RPC recommended as the primary path since it is guaranteed to exist on any self-hosted install; the REST API is a valid alternative but its availability should be verified first.
If we self-host, do we have to build our own OpenEduCat?	No – OpenEduCat is an existing free, open-source Odoo module that is installed, not written. Self-hosting only means running the Odoo + OpenEduCat server yourself instead of paying for hosting.
If a record is deleted on PIEAS, will the normal change-detection cycle catch it?	No – a filtered <code>last_updated</code> query only ever sees rows that still exist, so it structurally cannot notice an absence. A separate deletion cycle is required (Section 5).
If the Extractor already fetches changed records, why does anything check with PIEAS again for deletions?	The change cycle and the deletion cycle ask PIEAS two different questions — “what changed recently” (filtered) versus “what currently exists in full” (unfiltered ID list). The second cannot be derived from the first, so a second, separate Extractor call is necessary.
Isn’t the Extractor Agent supposed to be the only one that extracts from PIEAS – so who does the deletion comparison?	The Extractor remains the only agent that contacts PIEAS, for both cycles. The comparison itself (matching the fetched ID list against <code>sync_mapping</code>) is done locally by the Orchestrator, which never contacts PIEAS directly.
Who actually holds the <code>sync_mapping</code> table, and who decides when something is missing?	The Orchestrator exclusively owns <code>sync_mapping</code> and is the only agent that reads or writes it. It also owns the decision of what action (create / update / archive) follows from a comparison.

Question	Resolution
Was a separate Monitor Agent needed for change/deletion detection?	No – an earlier iteration of this design included one, but its responsibilities entirely overlapped with the Orchestrator's. It was removed; the Orchestrator now owns both scheduling and comparison directly, with the Extractor as its only fetch mechanism.

9 Conclusion

This project addresses two coupled problems – a one-time bulk migration and an ongoing, agentic synchronization, including deletion handling – between a source system with no API (PIEAS) and a target ERP with a well-documented ORM and API surface (OpenEduCat). The final design uses four agents with strictly non-overlapping responsibilities: an **Extractor** that is the sole point of contact with PIEAS and holds no state or decision logic; a **Transformer** and **Loader** that are stateless workers; and an **Orchestrator** that is the only agent making decisions, the only agent owning the `sync_mapping` state store, and the only agent that decides what the Extractor should fetch and when. An earlier design that introduced a dedicated Monitor Agent was simplified away once it became clear its job was indistinguishable from the Orchestrator's.

Within incremental sync, two independently-scheduled cycles run through the same four agents: a frequent *change cycle* using a timestamp watermark, and a less-frequent *deletion cycle* using a full ID fetch compared locally against `sync_mapping` via a `LEFT JOIN`. Data is written into OpenEduCat through its Odoo-based ORM (via XML-RPC) rather than directly against PostgreSQL, preserving the validation and business logic that OpenEduCat depends on to keep records consistent, and avoiding writes that would otherwise fail silently.